TEMA 5.2 – JSON, el lenguaje ligero

El término "lenguajes de marcas" engloba un amplio conjunto de lenguajes. Cada uno de ellos tiene sus propias reglas y características. JSON es diferente, principalmente, por su sencillez. **JSON no tiene etiquetas** y está **definido por una pequeña y compacta sintaxis capaz de representar estructuras de datos jerárquicas** utilizando un reducido conjunto de tipos de datos.

Gracias a sus características JSON es un lenguaje ampliamente utilizado en múltiples entornos: ficheros de configuración, intercambio de datos entre aplicaciones y, especialmente, en el desarrollo de aplicaciones web.

Introducción JSON

JSON (*JavaScript Object Notation*), un **formato de intercambio de datos** ampliamente utilizado en aplicaciones web y móviles. JSON es el formato en el que JavaScript representa internamente sus objetos. A pesar de ello, su utilización está tan extendida, que se considera un **lenguaje independiente**; se basa en una **estructura de pares clave-valor** y se utiliza para representar datos de forma legible.

La sintaxis de JSON es muy simple y los **documentos son más ligeros que en otros lenguajes** como XML, siendo estas dos características importantes de su éxito. Los documentos escritos en JSON son extremadamente sencillos de procesar desde *JavaScript*, ya que es su lenguaje de programación natural. También resulta muy sencillo procesarlos en el resto de los lenguajes de programación que disponen de procesadores (prácticamente todos los lenguajes modernos cuentan con ellos).

Además de utilizarse como lenguaje de intercambio entre sistemas, JSON (o alguno de sus dialectos) se emplea para **almacenar información en bases de datos no relacionales**. Los mismos datos expresados en JSON suelen ocupar menos espacio que en otros lenguajes, lo que lo convierte en una solución más **eficiente para el almacenamiento de grandes volúmenes de información**.

Los siguientes bloques de código muestran los mismos datos representados en **XML** (**341 bytes**) y **JSON** (**274 bytes**)

JSON

```
{
    "consolas": [
        {
            "nombre": "XBox Series",
            "fabricante": "Microsoft",
            "anyo": 2020
        },
        {
            "nombre": "Play Station 5",
            "fabricante": "Sony",
            "anyo": 2020
        }
    ]
}
```

XML

```
<xml versión="1.0" encoding="UTF-8"?>
<consolas>
   <consola>
      <nombre>Xbox Series</nombre>
      <fabricante>Microsoft</fabricante>
      <anyo>2020</anyo>
   </consola>
   <consola>
      <nombre>Play Station 5</nombre>
      <fabricante>Sony</fabricante>
      <anyo>2020</anyo>
   </consola>
</consolas>
```

JSON vs. XML

Aunque ambos lenguajes se suelen utilizar a menudo para resolver problemas similares, las capacidades de ambos son muy diferentes.

Ventajas y desventajas JSON

- ✓ **Sintaxis concisa:** JSON utiliza una sintaxis sencilla basada en pares clave-valor, lo que lo hace muy legible y fácil de escribir.
- ✓ **Ligero:** los documentos JSON son más compactos que los equivalentes en XML, lo que los convierte en una excelente opción para aplicaciones donde el rendimiento y la eficiencia son fundamentales.
- ✓ **Nativa compatibilidad con *JavaScript*:** dado que JSON se originó en *JavaScript*, se integra perfectamente con este lenguaje y es muy utilizado en aplicaciones *frontend*.
- ✓ **Procesamiento eficiente:** debido a su estructura simple, el procesamiento de datos JSON es rápido y eficiente.

- × **Menos descriptivo:** aunque JSON es fácil de leer, no proporciona tanta información estructural como XML, lo que podría ser una limitación en algunos casos.
- × **Falta de comentarios estructurales:** JSON carece de una forma nativa de incluir comentarios en su estructura. Los comentarios son útiles para documentar partes específicas del código y proporcionar información adicional a los desarrolladores. La ausencia de esta funcionalidad puede dificultar la comprensión y el mantenimiento del código en casos donde los comentarios son esenciales.
- × **Limitaciones en la representación de datos no estructurados:** JSON destaca por su estructura basada en pares clave-valor, lo que lo hace ideal para representar datos estructurados. Sin embargo, cuando se trata de datos no estructurados o semiestructurados, como documentos complejos o contenido multimedia, JSON puede resultar poco eficiente.

Ventajas y desventajas XML

- ✓ **Flexibilidad:** XML permite definir tus propias etiquetas y estructuras, lo que lo hace altamente adaptable a diferentes tipos de datos y necesidades.
- ✓ **Descriptivo:** XML proporciona una descripción más detallada de la estructura de los datos, lo que puede ser útil en contextos donde se necesita mayor claridad.
- ✓ **Validación:** XML es compatible con la validación de documentos mediante esquemas, lo que ayuda a garantizar la integridad y calidad de los datos.

- × **Verboso:** XML tiende a ser más verboso que JSON, lo que puede hacer que los documentos sean más grandes y difíciles de leer.
- × **Procesamiento más lento:** debido a su estructura más compleja, el procesamiento de datos XML puede ser más lento en comparación con JSON.

- **Tamaño de archivo mayor:** en comparación con JSON, los documentos XML tienden a ser más grandes debido a su sintaxis y a la necesidad de incluir etiquetas de apertura y cierre para cada elemento. Esto puede llevar a un aumento en el consumo de ancho de banda y tiempo de transferencia, lo que podría ser una consideración importante en aplicaciones donde la velocidad de transferencia es crucial, como en servicios web o aplicaciones móviles.

Comparativa JSON-XML

La elección entre JSON vs XML depende en gran medida del contexto y los requisitos de cada proyecto. Algunas orientaciones de uso pueden ser:

Usar JSON si:

- Se desea lograr una estructura simple y ligera.
- Se trabaja con aplicaciones *frontend* y se necesita una integración fluida con *JavaScript*.
- La eficiencia y el rendimiento son cruciales.

Se recomienda usar JSON en entornos en los que **la robustez de los datos esté garantizada de partida**, es decir, no sea determinante; ya que JSON puede ser una opción más eficiente. Igualmente, JSON será la opción ideal en entornos en los que **el volumen de transferencia de datos sea muy alto**, siempre que la fiabilidad de estos no esté comprometida o no sea un factor crítico.

Usar XML si:

- Se necesita alcanzar una descripción detallada de la estructura de los datos.
- Se requiere validar los documentos para garantizar la integridad de los datos.
- Se trabaja en contextos donde la flexibilidad para definir etiquetas personalizadas es esencial.

Se recomienda usar XML en entornos en los que la **fiabilidad de los datos es un factor relevante**, debido principalmente a la existencia de mecanismos de validación con las implicaciones que estos tienen sobre las relaciones, los tipos de datos o las cardinalidades de elementos.

En definitiva, **no existe un lenguaje mejor que otro, sino una más o menos adecuado para cada uso concreto**. Si se valora la simplicidad y la eficiencia, JSON podrá ser le elección. Si se busca una descripción más rica y validación de datos, XML será la opción adecuada.

Uso de JSON

Existen varias razones por las que se utiliza JSON:

- **Independencia del lenguaje de programación:** JSON es independiente del lenguaje de programación utilizado en tu sistema, a pesar de estar derivado de *JavaScript*.
- **Representación universal de datos:** JSON representa datos que comparten elementos comunes de muchos lenguajes de programación, lo que lo convierte en una forma universal de representación de datos comprensible por todos los sistemas.
- **Legibilidad:** JSON es legible por humanos cuando se formatea adecuadamente, lo que facilita la comprensión y el análisis de los datos.
- **Compacidad:** El formato de datos JSON no utiliza una estructura de marcado completa como XML, lo que lo hace más compacto y eficiente en términos de tamaño de archivo y uso de ancho de banda.
- **Facilidad de análisis:** JSON se puede desglosar fácilmente en componentes sintácticos lógicos, especialmente en entornos de programación como *JavaScript*.
- **Disponibilidad de bibliotecas:** Existen numerosas bibliotecas de JSON disponibles para la mayoría de los lenguajes de programación, lo que facilita su implementación y manipulación en diferentes entornos de desarrollo.

Ejemplo JSON

El siguiente ejemplo muestra un XML y su correspondencia con un documento JSON:

JSON

```
{
   "alumnos": {
      "alumno": [
         {
            "nombre": "Juan",
            "apellido": "Pérez"
         },
         {
            "nombre": "María",
            "apellido": "Gómez"
         },
         {
            "nombre": "Pedro",
            "apellido": "López"
         }
      ]
   }
}
```

XML

```
<alumnos>
  <alumno>
    <nombre>Juan</nombre>
    <apellido>Pérez</apellido>
  </alumno>
  <alumno>
    <nombre>María</nombre>
    <apellido>Gómez</apellido>
  </alumno>
  <alumno>
    <nombre>Pedro</nombre>
    <apellido>López</apellido>
  </alumno>
</alumnos>
```

La estructura de JSON puede ser diferente de la de XML, y a menudo no es necesario utilizar etiquetas adicionales como en XML. En JSON, la estructura de datos es más flexible y, como se pude ver, no es necesario agregar una etiqueta adicional para cada ***alumno***.

El ejemplo anterior se puede simplificar eliminando el objeto ***alumnos***:

```
{
  "alumno": [
    {"nombre": "Juan", "apellido": "Pérez"},
    {"nombre": "María", "apellido": "Gómez"},
    {"nombre": "Pedro", "apellido": "López"}
  ]
}
```

Además, se puede alcanzar un grado final de simplificación, en el que dependiendo de cómo estén estructurado el código que usa el fichero JSON, podría no necesitar la clave ***"alumno"*** para acceder a los datos:

```
[
   {"nombre": "Juan", "apellido": "Pérez"},
   {"nombre": "María", "apellido": "Gómez"},
   {"nombre": "Pedro", "apellido": "López"}
]
```

Estructura de un documento JSON

La estructura básica de un documento JSON consiste en **pares clave-valor encerrados entre llaves** `{}`.

- Cada clave se representa como una cadena de caracteres y se separa de su valor por dos puntos (`:`).
- Los pares clave-valor se separan entre sí por comas (`,`). Para evitar problemas, se omite la coma después del último elemento en la lista de pares.

Valores de las claves

El valor de las claves puede ser de diferentes tipos:

- **Número** (entero o de coma flotante, los números flotantes utilizan el punto como separador decimal).
- **Cadena** (entre comillas dobles (`""`)).
- **Booleano** (`true`/`false`).
- **Matriz** (entre corchetes (`[ ]`); sus elementos se separan mediante comas).
- **Objeto** (entre llaves (`{}`)).
- **Nulo** (`null`).

Ejemplo:

```
{
   "numero": 42,
   "flotante": 9.99,
   "cadena": "Hola Mundo",
   "booleano": true,
   "matriz": [1, 2, 3, 4, 5],
   "objeto": {
      "clave": "valor",
      "subobjeto": {
          "atributo": "dato"
      }
   },
"nulo": null
}
```

En este ejemplo, se muestra un número entero con la clave «numero», un número flotante con la clave «flotante», una cadena de texto con la clave «cadena», un valor booleano `true` con la clave «booleano», una matriz de números con la clave «matriz», un objeto con la clave «objeto» que contiene subclaves y valores, y finalmente, el valor nulo con la clave «nulo».

Estructura de un documento JSON: Inicio con { o [

La estructura básica de un JSON comienza con un símbolo distintivo que indica si el JSON representa un **objeto** o una **matriz**. La elección entre llaves {} o corchetes [] determina el tipo principal de estructura que se está utilizando.

En JSON, solo puede haber un único objeto raíz, y este debe comenzar con llaves {} o corchetes [], dependiendo de la estructura principal.

JSON de Objeto (encerrado entre llaves {})

Cuando un **JSON empieza con llaves**, representa un **objeto**. Un objeto es una colección de pares clave-valor donde cada clave es una cadena de caracteres. El valor asociado a cada clave puede ser de diversos tipos, como números, cadenas, booleanos, matrices, objetos o *null*.

Ejemplo JSON de Objeto

```
{
   "titulo": "El Principito",
   "autor": "Antoine de Saint-Exupéry",
   "publicacion": 1943,
   "valoraciones": [4.8, 4.9, 5.0],
   "resenia": {
      "resumen": "La historia de un pequeño príncipe ...",
      "opiniones": [
         {"usuario": "Lector123", "puntuacion": 5},
         {"usuario": "Viajero789", "puntuacion": 4.5}
      ]
   },
   "premios": null
}
```

JSON de Matriz (encerrado entre corchetes [])

Cuando un **JSON empieza con corchetes**, representa una **matriz**. Una matriz es una colección ordenada de valores, donde cada elemento puede ser de cualquier tipo permitido en JSON.

Ejemplo JSON de Matriz

```
[
  {
    "titulo": "Inception",
    "director": "Christopher Nolan",
    "lanzamiento": 2010,
    "genero": ["Acción", "Ciencia Ficción"]
  },
  {
    "titulo": "The Shawshank Redemption",
    "director": "Frank Darabont",
    "lanzamiento": 1994,
    "genero": ["Drama"]
  }
]
```

Elección entre un objeto ({}) y una matriz ([])

La elección entre un objeto ({}) y una matriz ([]) en JSON depende de la naturaleza de los datos que se están representando y de la estructura que mejor se ajuste a esos datos. Veamos algunas consideraciones comunes sobre cómo y por qué se toma esa decisión:

Objeto ({})

- Se elige un objeto cuando los datos pueden organizarse de manera lógica en pares clave-valor.
- Cada propiedad (clave) en el objeto describe una característica específica, y su valor asociado puede ser de cualquier tipo de dato permitido en JSON, incluyendo números, cadenas, booleanos, objetos, matrices o *null*.
- Útil para representar entidades individuales con propiedades distintas.

Ejemplo de Objeto

```
{
    "nombre": "Juan",
    "edad": 25,
    "ciudad": “Madrid”
}
```

Matriz ([])

- Se elige una matriz cuando se tiene una lista ordenada de elementos del mismo tipo.
- Cada elemento de la matriz puede ser de cualquier tipo de dato permitido en JSON, incluyendo números, cadenas, booleanos, objetos, matrices o *null*.
- Útil para representar conjuntos de datos como listas o colecciones.

Ejemplo de Objeto

```
[
    “Manzanas”,
    “Peras”,
    “Sandias”
]
```

Ejemplos combinados

Objeto anidado en matriz

```
[
   {"nombre": "Juan", "edad": 25},
   {"nombre": "María", "edad": 30},
   {"nombre": "Carlos", "edad": 22}
]
```

En el ejemplo cada elemento de la matriz es un objeto que representa a una persona.

Objeto anidado en matriz con dos niveles

```
{
    "personas": [
      {"nombre": "Juan", "edad": 25},
      {"nombre": "María", "edad": 30},
      {"nombre": "Carlos", "edad": 22}
    ]
 }
```

En el ejemplo un objeto contiene una matriz de objetos, organizando la información de personas..

Objeto anidado en matriz con varios niveles

```
{
  "personas": [
    {
      "persona": {
        "nombre": "Juan",
        "edad": 25
      }
    },
    {
      "persona": {
        "nombre": "María",
        "edad": 30
      }
    },
    {
      "persona": {
        "nombre": "Carlos",
        "edad": 22
      }
    }
  ]
}
```

La elección entre objeto y matriz depende de la estructura y la relación de los datos que se están representando, así como de las necesidades específicas de la aplicación o contexto en el que se utiliza JSON. Es recomendable evitar niveles innecesarios en la estructura del JSON. Mantener el diseño simple facilita el trabajo con el documento, mejora la legibilidad y puede optimizar el rendimiento en aplicaciones que manejan datos JSON.

Representación de un atributo XML en un JSON

En XML, los atributos se representan dentro de una etiqueta utilizando la sintaxis `nombre="valor"`. Sin embargo, en JSON, la estructura es diferente y **no hay una representación directa de atributos**. En JSON, los datos se almacenan como pares clave-valor, y los atributos de XML a menudo se representan como propiedades en el objeto.

Ejemplo de XML con atributos

```
<alumno id="1" curso="A">
   <nombre>Juan</nombre>
   <apellido>Pérez</apellido>
</alumno>
```

Representación en JSON

```
{
   "alumno": {
   "id": 1,
   "curso": "A",
   "nombre": "Juan",
   "apellido": "Pérez"
   }
}
```

En este ejemplo, los atributos *"id"* y *"curso"* se han convertido en propiedades dentro del objeto JSON correspondiente. Cada propiedad en el objeto JSON es una clave seguida de un valor. Este enfoque mantiene la estructura de pares clave-valor que es común en JSON.

Es importante señalar que esta es una **convención común**, pero **no hay una regla estricta sobre cómo convertir atributos XML a JSON**. La elección puede depender de la estructura específica de tus datos y de cómo prefieras organizar la información en JSON.

Otra conversión habitual, en el caso de que se quiera dar a conocer que una propiedad JSON se corresponde con un atributo XML, es **colocar un guion (-) o una barra baja (_) delante de la clave** asociada a la propiedad JSON.

Representación en JSON

```
{
   "alumno": {
   "-id": 1,
   "_curso": "A",
   "nombre": "Juan",
   "apellido": "Pérez"
   }
}
```

RESUMEN

Sintaxis de los objetos en JSON

A continuación se presenta una tabla con las posibles combinaciones de pares clave-valor:

Tipo	Ejemplo
Un único par tipo-valor, siendo el valor una cadena de caracteres.	`{"titulo":"Los Ojos de Julia"}`
Un único par tipo-valor, siendo el valor un número entero.	`{"anyo":2010}`
Un único par tipo-valor, siendo el valor un número decimal.	`{"calificacion":6.1}`
Un único par tipo-valor, siendo el valor de tipo lógico.	`{"estreno-cine":true}`
Dos pares tipo-valor.	`{ "titulo":"Los Ojos de Julia", "anyo":2010 }`
Tres pares tipo-valor, siendo el último de ellos un objeto.	`{ "titulo":"Los Ojos de Julia", "anyo":2010, "director":{ "nombre":"Guillem Morales", "nacionalidad":"Español" } }`

Sintaxis de las listas en JSON

En su versión más sencilla las listas pueden contener colecciones de valores simples:

```
{
    "titulo" : "Nosferatu, vampiro de la noche",
    "titulo-original" : "Nosferatu: Phantom der Nacht",
    "director" : "Werner Herzog",
    "reparto" : ["Klaus Kinski","Bruno Ganz","Jacques Dufilho"]
}
```

En su versión más elaborada las listas pueden contener colecciones de objetos:

```
{
    "titulo":"Los Ojos de Julia",
    "anyo":2010,
    "director":{
        "nombre":"Guillem Morales",
        "nacionalidad":"Español"
    },
    "reparto":[
        {
            "nombre":"Belén Rueda",
            "personaje":"Julia"
        },
        {
            "nombre":"Lluis Homar",
            "personaje":"Isaac"
        },
        {
            "nombre":"Francesc Orella",
            "personaje":"Inspector Dimas"
        }
    ]
}
```

Ejercicios resueltos

Información de productos

Partiendo del siguiente código XML, que represente la información de distintos productos, elabora un documento JSON que permita representar la misma información.

Código XML

```
<productos>
   <producto>
      <nombre>Camiseta</nombre>
      <precio>19.99</precio>
      <disponible>true</disponible>
      <detalles>
         <color>Rojo</color>
         <talla>M</talla>
      </detalles>
   </producto>
   <producto>
      <nombre>Pantalón</nombre>
      <precio>29.99</precio>
      <disponible>false</disponible>
      <detalles>
         <color>Azul</color>
         <talla>L</talla>
    </detalles>
   </producto>
</productos>
```

Código JSON (existen múltiples representaciones posibles, esta es la más usual)

```
{
  "productos": [
    {
      "nombre": "Camiseta",
      "precio": 19.99,
      "disponible": true,
      "detalles": {
        "color": "Rojo",
        "talla": "M"
      }
    },
    {
      "nombre": "Pantalón",
      "precio": 29.99,
      "disponible": false,
      "detalles": {
        "color": "Azul",
        "talla": "L"
      }
    }
  ]
}
```

JSON con errores I

Un array no puede tener pares clave-valor, los pares clave-valor conforman objetos.

```
[
  "lenguaje": "JavaScript",
  "paradigma": "Multiparadigma",
  "popularidad": 1,
  "usos": ["Desarrollo web", "Aplicaciones móviles", "Back-end"]
]
```

- El array (**[ ]**) no puede contener pares clave-valor directamente.
- Los pares clave-valor deben estar dentro de un objeto (**{}**).

Corrección

Si la intención es que **el array contenga un solo objeto**:

```
[
    {
        "lenguaje": "JavaScript",
        "paradigma": "Multiparadigma",
        "popularidad": 1,
        "usos": ["Desarrollo web", "Aplicaciones móviles", "Back-end"]
    }
]
```

Si **el array debe contener varios objetos**:

```
[
    {
        "lenguaje": "JavaScript",
        "paradigma": "Multiparadigma",
        "popularidad": 1,
        "usos": ["Desarrollo web", "Aplicaciones móviles", "Back-end"]
    },
    {
        "lenguaje": "Python",
        "paradigma": "Multiparadigma",
        "popularidad": 2,
        "usos": ["IA", "Desarrollo web", "Automatización"]
    }
]
```

JSON con errores II

JSON permite un único objeto raíz.

```
{
  "lenguaje": "JavaScript",
  "paradigma": "Multiparadigma",
  "popularidad": 1,
  "usos": ["Desarrollo web", "Aplicaciones móviles", "Back-end"]
},
{
  "leguaje": "Python",
  "paradigma": "Multiparadigma",
  "popularidad": 2,
  "usos": ["IA", "Desarrollo web", "Automatización"]
}
```

Corrección

- JSON con un nivel más, para que **exista un objeto raíz**:

```
[
  {
     "lenguaje": {
        "nombre": "JavaScript",
        "paradigma": "Multiparadigma",
        "popularidad": 1,
        "usos": ["Desarrollo web", "Aplicaciones móviles", "Back-end"]
     }
  },
  {
     "lenguaje": {
        "nombre": "Python",
        "paradigma": "Multiparadigma",
        "popularidad": 2,
        "usos": ["IA", "Desarrollo web", "Automatización"]
     }
  }
]
```

- JSON con dos niveles más:

```
{
  "lenguajes": [
    {
      "lenguaje": {
        "nombre": "JavaScript",
        "paradigma": "Multiparadigma",
        "popularidad": 1,
        "usos": ["Desarrollo web", "Aplicaciones móviles", "Back-end"]
      }
    },
    {
      "lenguaje": {
        "nombre": "Python",
        "paradigma": "Multiparadigma",
        "popularidad": 2,
        "usos": ["IA", "Desarrollo web", "Automatización"]
      }
    }
  ]
}
```

Actividad resuelta 7.1: Representación de un lenguaje de programación

Representar en un fichero JSON un lenguaje de programación con la siguiente información: nombre del lenguaje, año creación, nombre de los autores e indicador de si es compilado o no.

Solución

```
{
    "nombre":"C",
    "anyo-presentacion":1972,
    "creadores":"Dennis Ritchie",
    "compilado":true
}
```

Actividad resuelta 7.2: Representación de un lenguaje de programación

Representar en un fichero JSON un equipo de baloncesto con la siguiente información: nombre del equipo, color de la equipación, nombre del estadio, nombre del entrenador, indicador de si la plantilla del equipo se corresponde con una plantilla histórica, referencia a su página web (si está disponible) y jugadores del mismo (de cada jugador se almacena su nombre y su dorsal; además, estos se agrupan según las siguientes posiciones: *base*, *alero* y *pivot*).

Solución

```
{
    "nombre":"Los Angeles Lakers",
    "colores":"Púrpura y Oro",
    "estadio":"Staples Center",
    "entrenador":"Phil Jackson",
    "historico":true,
    "web":null,
    "bases":[
        {
            "nombre":"Magic Johnson",
            "dorsal":32
        },
        {
            "nombre":"Jerry West",
            "dorsal":44
        }
    ],
    "aleros":[
        {
            "nombre":"James Worthy",
            "dorsal":42
        },
        {
            "nombre":"Kobe Bryant",
            "dorsal":8
        }
    ],
   "pivotes":[
        {
            "nombre":"Kareem Abdul-Jabbar",
            "dorsal":33
        },
        {
            "nombre":"Wilt Chamberlain",
            "dorsal":13
        },
        {
            "nombre":"Pau Gasol",
            "dorsal":16
        }
    ]
}
```

Actividad resuelta 7.3: Configuración de aplicaciones

Prácticamente todas las aplicaciones permiten crear configuraciones personalizadas para los usuarios que las utilizan. Estos pueden establecer los valores apropiados para algunos parámetros y adaptar los sistemas a sus gustos o necesidades.

Suponiendo que la aplicación permite configurar el color de fondo, el tamaño del texto, el color del texto, el idioma y la pantalla de inicio; a continuación se presenta una posible estructura del fichero JSON que permitiría soportar estas configuraciones:

```
{
    "usuarios" : [
        {
            "identificador":"jhipolito",
            "configuracion": {
                    "color-fondo":"#AA99AA",
                    "tamaño-texto":12,
                    "color-texto":"#441238",
                    "idioma":"es",
                    "pantalla-inicio":"listado-clientes"
            }
        },
        {
            "identificador":"alanchas",
            "configuracion": {
                    "color-fondo":"#8911F8",
                    "tamaño-texto":14,
                    "color-texto":"#311103",
                    "idioma":"it",
                    "pantalla-inicio":"listado-deudores"
            }
        }
    ]
}
```

Actividades propuestas

Actividad propuesta 7.1: Representación de una obra pictórica

Crear un documento JSON que permita representar un cuadro con la siguiente información: título del cuadro, nombre del autor, año de creación, indicador de si está expuesta en un museo y nombre del museo (puede ser nulo).

Actividad propuesta 7.2: Representación de la obra de un pintor

Crear un documento JSON que representa la obra de un pintor. El documento debe almacenar la siguiente información: nombre del pintor, fecha de nacimiento, lugar de nacimiento, fecha de defunción (puede ser nula), lugar de defunción (puede ser nulo) y listado de obras que contendrá por cada obra el título del cuadro, el año de creación y la URL a una fotografía del cuadro alojada en internet.

Actividad propuesta 7.3: Detección de errores

Encuentra los errores de sintaxis que presenta el siguiente documento JSON:

```
{
    "pelicula": "El Resplandor",
    "director": "Stanley Kubrick"
    "estreno"; 1980,
    "reparto": [
        {
            "nombre": "Jack Nicholson",
            "personaje": "Jack Torrance"
        },
        {
            "nombre": "Shelley Duvall",
            "personaje": "Wendy Torrance"
        },
        {
            "nombre": "Danny Lloid",
            "personaje": "Danny Torrance"
    ]
}
```